AI ASSISTED CODING

LAB-17.5

Katta Lasya

2303A51724

Batch-11

Task 1 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

- **Handle missing values in loan_amount, credit_score.**
- **Encode loan_status (Approved/Rejected).**
- **Scale numerical features using standard scaling.**
- **Split the dataset into training and testing sets.**

Sample Code:

```
import pandas as pd
data = pd.read_csv("loadapproval.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/amineipad/loan-approval-dataset

Expected Output:

- **A machine-learning-ready loan dataset.**

PROMPT:

#Generate preprocessing code for the loan_approval_dataset.csv dataset.

"""Handle missing values in loan_amount, credit_score.

Encode loan_status (Approved/Rejected).

Scale numerical features using standard scaling.

Split the dataset into training and testing sets."""

GIVEN CODE AND OUTPUT:

AIACLab17.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[3]

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
import numpy as np

# Load the dataset
df = pd.read_csv('/content/loan_approval_dataset.csv')
display(df.head())
```

	loan_id	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value
0	1	2	Graduate	No	9600000	29900000	12	778	2400000	17600000
1	2	0	Not Graduate	Yes	4100000	12200000	8	417	2700000	2200000
2	3	3	Graduate	No	9100000	29700000	20	506	7100000	4500000
3	4	3	Graduate	No	8200000	30700000	8	467	18200000	3300000
4	5	5	Not Graduate	Yes	9800000	24200000	20	382	12400000	8200000

Handle Missing Values

We will fill missing `loan_amount` and `credit_score` values with the median of their respective columns to avoid skewing the distribution with the mean and to maintain the number of samples.

Gemini

[9]

```python
# Strip whitespace from column names to handle potential incc
df.columns = df.columns.str.strip()

-# Fill missing 'loan_amount' and 'credit_score' with their me
+# Fill missing 'loan_amount' and 'cibil_score' with their med
df['loan_amount'] = df['loan_amount'].fillna(df['loan_amount'
-df['credit_score'] = df['credit_score'].fillna(df['credit_scc
+df['cibil_score'] = df['cibil_score'].fillna(df['cibil_score'

print("Missing values after handling:")
-print(df[['loan_amount', 'credit_score']].isnull().sum())
+print(df[['loan_amount', 'cibil_score']].isnull().sum())
```

```
Missing values after handling:
loan_amount    0
cibil_score    0
dtype: int64
```

Encode `loan_status`

We will convert the `loan_status` column from categorical ('Approved', 'Rejected') to numerical (1, 0) using `LabelEncoder`.

[5]

```python
# Encode 'loan_status' (Approved: 1, Rejected: 0)
le = LabelEncoder()
df['loan_status_encoded'] = le.fit_transform(df['loan_status'])

print("Loan Status Mapping:", dict(zip(le.classes_, le.transform(le.classes_))))
display(df[['loan_status', 'loan_status_encoded']].head())
```

```
Loan Status Mapping: {' Approved': np.int64(0), ' Rejected': np.int64(1)}
```

	loan_status	loan_status_encoded
0	Approved	0
1	Rejected	1
2	Rejected	1
3	Rejected	1
4	Rejected	1

Scale Numerical Features

Numerical features (`loan_amount`, `credit_score`, etc.) will be scaled using `StandardScaler` to standardize them, which is crucial for many machine learning algorithms.

```python
# Identify numerical features for scaling (excluding the ID and target columns)
numerical_cols = df.select_dtypes(include=np.number).columns.tolist()
numerical_cols = [col for col in numerical_cols if col not in ['loan_id', 'loan_status_encoded']]

# Initialize StandardScaler
scaler = StandardScaler()

# Apply scaling to numerical features
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

display(df[numerical_cols].head())
```

	no_of_dependents	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value
0	-0.294102	1.617979	1.633052	0.192617	1.032792	-0.780058	2.877289	0.832028	0.930304
1	-1.473548	-0.341750	-0.324414	-0.508091	-1.061051	-0.733924	-0.631921	-0.694993	-0.515936
2	0.295621	1.439822	1.610933	1.594031	-0.544840	-0.057300	-0.107818	1.996520	2.407316
3	0.295621	1.119139	1.721525	-0.508091	-0.771045	1.649637	-0.381263	0.897943	0.899533
4	1.475067	1.689242	1.002681	1.594031	-1.264055	0.757724	0.735304	1.568075	0.007172

Split the Dataset

Finally, the dataset will be split into training and testing sets to prepare for model training and evaluation. We'll use `loan_status_encoded` as the target variable.

```python
# Define features (X) and target (y)
X = df.drop(['loan_id', 'loan_status', 'loan_status_encoded'], axis=1)
y = df['loan_status_encoded']

# Split the dataset into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

print(f"X_train shape: {X_train.shape}")
print(f"X_test shape: {X_test.shape}")
print(f"y_train shape: {y_train.shape}")
print(f"y_test shape: {y_test.shape}")

display(X_train.head())
```

```
X_train shape: (3415, 11)
X_test shape: (854, 11)
y_train shape: (3415,)
y_test shape: (854,)
```

	no_of_dependents	education	self_employed	income_annum	loan_amount	loan_term	cibil_score	residential_assets_value	commercial_assets_value	luxury_assets_value	bank_asset_value
1224	-1.473548	Graduate	Yes	1.689242	0.969504	0.542970	-0.817446	-0.995347	1.920232	0.656256	0.684136
478	0.295621	Not Graduate	Yes	-0.519907	-0.722543	-0.508091	1.102394	-0.318723	0.211201	-0.585136	0.068714
3065	-0.294102	Not Graduate	No	-1.090010	-1.231263	0.542970	-0.568041	-0.395612	-0.791431	-1.068509	-1.131358
326	-0.294102	Not Graduate	Yes	-0.413012	-0.423946	0.192617	-0.486839	-0.672413	-1.133237	-0.519221	-0.115912
2991	0.295621	Not Graduate	No	-0.983115	-0.733602	0.192617	-0.249034	-0.595524	-0.814218	-0.980623	-1.131358

Task 2 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

- **Remove duplicate user profiles.**
- **Handle missing values**
- **Encode categorical columns**

Dataset link:

https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles

Expected Output:

- **A cleaned dataset for engagement prediction.**

PROMPT:

Generate a preprocessing python code for SocialMediaTop100.csv and Remove duplicate user profiles.

Handle missing values

Encode categorical columns

GIVEN CODE AND OUTPUT:

```python
import pandas as pd
from sklearn.preprocessing import LabelEncoder
import numpy as np

# Load the dataset with 'latin1' encoding
social_df = pd.read_csv('/content/SocialMediaTop100.csv', encoding='latin1')
print("Original DataFrame Info:")
social_df.info()
display(social_df.head())
```

```
Original DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   N          500 non-null    int64
 1   PROFILE    500 non-null    object
 2   FOLLOWERS  500 non-null    float64
 3   POSTS      400 non-null    float64
 4   Platform   500 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 19.7+ KB
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

Remove Duplicate User Profiles

We will check for a 'Username' column to identify and remove duplicate user profiles. If 'Username' is not present, we will remove duplicates based on all columns.

```python
if 'Username' in social_df.columns:
    initial_rows = social_df.shape[0]
    social_df.drop_duplicates(subset=['Username'], inplace=True)
    print(f"Removed {initial_rows - social_df.shape[0]} duplicate user profiles based on 'Username'.")
else:
    initial_rows = social_df.shape[0]
    social_df.drop_duplicates(inplace=True)
    print(f"Removed {initial_rows - social_df.shape[0]} duplicate rows based on all columns (no 'Username' column found).")

print("DataFrame after removing duplicates:")
social_df.info()
```

```
Removed 0 duplicate rows based on all columns (no 'Username' column found).
DataFrame after removing duplicates:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   N          500 non-null    int64
 1   PROFILE    500 non-null    object
 2   FOLLOWERS  500 non-null    float64
 3   POSTS      400 non-null    float64
 4   Platform   500 non-null    object
dtypes: float64(2), int64(1), object(2)
memory usage: 19.7+ KB
```

Handle Missing Values

We will fill missing numerical values with the median and missing categorical values with the mode of their respective columns.

```python
for col in social_df.columns:
    if social_df[col].isnull().any():
        if pd.api.types.is_numeric_dtype(social_df[col]):
            median_val = social_df[col].median()
            social_df[col].fillna(median_val, inplace=True)
            print(f"Filled missing values in numerical column '{col}' with median: {median_val}")
        elif pd.api.types.is_object_dtype(social_df[col]) or pd.api.types.is_categorical_dtype(social_df[col]):
            mode_val = social_df[col].mode()[0]
            social_df[col].fillna(mode_val, inplace=True)
            print(f"Filled missing values in categorical column '{col}' with mode: {mode_val}")

print("\nMissing values after handling:")
print(social_df.isnull().sum())
```

```
Filled missing values in numerical column 'POSTS' with median: 2470.5

Missing values after handling:
N            0
PROFILE      0
FOLLOWERS    0
POSTS        0
Platform     0
dtype: int64
```

Encode Categorical Columns

We will use `LabelEncoder` for categorical columns to convert them into numerical representations. This is suitable for many machine learning models.

```python
le = LabelEncoder()

for col in social_df.select_dtypes(include=['object', 'category']).columns:
    # Exclude columns that are clearly identifiers or might not need encoding if they exist
    if col not in ['Username']:
        social_df[col] = le.fit_transform(social_df[col])
        print(f"Encoded categorical column '{col}'. Unique values after encoding: {social_df[col].unique()}")

print("\nDataFrame after encoding categorical columns:")
social_df.info()
display(social_df.head())
```

```
Encoded categorical column 'PROFILE'. Unique values after encoding: [151  86 204 216 323 105 193  33  59 190 173 186 265 271 351 164 383  55
 199 246 261 178 187 405  95 416   2 110 302 422  45 118  43 375  89 129
 382 258 293 208 189 327 456 135  93 331 381 203 266 259 329 263 176  21
  94 367 311 115  48 184 239 174 421 392 299  29 228  18 156 158 283 297
 230  31 390 291   1 231 247 182  77 177 284 407 217 175 401  56 377 349
 318 308 341 212 426 153 324 400 250  38  54 306 111 399  66 374  60 267
  63 355 214 120  40 280  26   8  64 277 325 268 107  49 205 103 142  41
 317 393 225 220 167 218  76 145 279 144 275 464 463  25 191  23 181  84
 281  16 354 200 137 290  42  35 269  28 304 292 334  10  82  30 384 237
 125 244 233 446  75  22 300 310 309 345  81 314 286 255   7 219 388 403
 386   0  44 136 333  39 172 141  69 404 330 289 104  79 253 238  83 389
 114  14 365 332 346 108  32 224 395 350 170  53 150 123 121  47 459 185
 387  19 134  11  13 428 305 221 227   5 359 210 128 274 194 326  90  73
 236 454 453 347 342 240 378 398 235  12 245 369 232 313 316 180 149 211
 127 322 226 130 254  67 357 102 452 336 328  52  37 358 165 376 433 413
  61 272 414 312 352 455 360 449 457 364 260  17 410 106 251  65 443 372
 133 109 391 126 301 462 288 315 344 119 139 337 131 424  80 436 112  36
  27  20 444 420 248 295 371 147 252 207 434 417 270 339 441 408 209 435
 338 373 117 303 285 249 450  71 370 397 223  97 445 282 461 412 278 380
 152 160  91 264 143 321 132  70 361 348  24 234 113  99 124 430 241 257
 140 159 287  85 379 439 406 431 419 432 423 188  57  15 402 195 363 425
 427 335 100 356 162 243  50 396 307   9 201  62 171 166 415   6 168 296
  72 448 256   4 161  51 183 458 447 460 242 138 438 146 385  46 340 294
  58 192 222  87 418 197 320 368  34 198 122 155 429 140 440 273 366 262
  68 353 411 451 179 215 196 276 163  92  88 202 116 206 169 298  74 394
 409 213 362  98  96 101 343 319 437 229 154  78 157 442   3]
Encoded categorical column 'Platform'. Unique values after encoding: [0 3 4 2 1]

DataFrame after encoding categorical columns:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 5 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   N          500 non-null    int64
 1   PROFILE    500 non-null    int64
 2   FOLLOWERS  500 non-null    float64
 3   POSTS      500 non-null    float64
 4   Platform   500 non-null    int64
dtypes: float64(2), int64(3)
memory usage: 19.7 KB
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	151	539446645.0	7202.0	0
1	2	86	473864939.0	3338.0	0
2	3	204	364542529.0	6935.0	0
3	4	216	355790796.0	890.0	0
4	5	323	341579063.0	1828.0	0

Task 3 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

- **Convert date column into datetime format.**
- **Handle missing values in temperature, humidity.**
- **Create new features (season, week_number).**
- **Normalize rainfall values.**

Dataset link:

https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input

Expected Output:

- **A processed dataset ready for forecasting models.**

PROMPT:

Write a Python code using pandas and numpy to preprocess a weather dataset for forecasting. Convert the date column into datetime format, handle missing values in temperature and humidity, create new features like season and week number, and normalize the rainfall column. The final output should be a clean dataset ready for forecasting models.

GIVEN CODE AND OUTPUT:

Weather Data Preprocessing for Forecasting

This notebook demonstrates the preprocessing steps for a weather dataset, preparing it for use in forecasting models. We will cover loading data, handling missing values, feature engineering, and normalization.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Load the dataset
try:
    df = pd.read_csv('/content/weather_forecast_data.csv')
    print('Dataset loaded successfully. First 5 rows:')
    display(df.head())
except FileNotFoundError:
    print("Error: 'weather_forecast_data.csv' not found. Please ensure the file is in the correct directory.")
    df = pd.DataFrame() # Create an empty DataFrame to avoid further errors
```

```
Dataset loaded successfully. First 5 rows:
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

1. Convert Date Column to Datetime Format

We convert the 'Date' column to a datetime object, which is essential for time-series analysis and feature extraction.

```python
# Convert 'Date' column to datetime objects
if 'Date' in df.columns:
    df['Date'] = pd.to_datetime(df['Date'])
    print('\'Date\' column converted to datetime format. Info:')
    df.info()
else:
    print("Warning: 'Date' column not found. Skipping date conversion.")
```

```
Warning: 'Date' column not found. Skipping date conversion.
```

2. Handle Missing Values in Temperature and Humidity

For numerical columns like 'Temperature (C)' and 'Humidity (%)', we'll fill missing values using the mean of their respective columns. This is a common imputation strategy that maintains the overall distribution.

```python
# Handle missing values for 'Temperature' and 'Humidity'
if 'Temperature' in df.columns:
    df['Temperature'] = df['Temperature'].fillna(df['Temperature'].mean())
    print("Missing values in 'Temperature' filled with mean.")
else:
    print("Warning: 'Temperature' column not found. Skipping imputation.")

if 'Humidity' in df.columns:
    df['Humidity'] = df['Humidity'].fillna(df['Humidity'].mean())
    print("Missing values in 'Humidity' filled with mean.")
else:
    print("Warning: 'Humidity' column not found. Skipping imputation.")

print('\nDataFrame after handling missing values (first 5 rows):')
display(df.head())
```

```
Missing values in 'Temperature' filled with mean.
Missing values in 'Humidity' filled with mean.

DataFrame after handling missing values (first 5 rows):
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

3. Create New Features: Season and Week Number

- **Season**: We'll define seasons based on months (e.g., December-February as Winter, March-May as Spring, etc.).
- **Week Number**: Extract the week of the year from the date, which can capture weekly patterns.

```python
if 'Date' in df.columns:
    # Create 'Season' feature
    def get_season(date):
        month = date.month
        if 3 <= month <= 5: # March, April, May
            return 'Spring'
        elif 6 <= month <= 8: # June, July, August
            return 'Summer'
        elif 9 <= month <= 11: # September, October, November
            return 'Autumn'
        else: # December, January, February
            return 'Winter'
    df['Season'] = df['Date'].apply(get_season)
    print("\'Season\' feature created.")

    # Create 'Week_Number' feature
    df['Week_Number'] = df['Date'].dt.isocalendar().week.astype(int)
    print("\'Week_Number\' feature created.")

    print('\nDataFrame with new features (first 5 rows):')
    display(df.head())
else:
    print("Skipping feature creation as 'Date' column is missing.")
```

```
Skipping feature creation as 'Date' column is missing.
```

4. Normalize the Rainfall Column

Normalization (Min-Max scaling) transforms the 'Rainfall (mm)' column so its values fall within a specific range (usually 0 to 1). This is often beneficial for machine learning models.

```python
# Normalize 'Rain' column
if 'Rain' in df.columns:
    if pd.api.types.is_numeric_dtype(df['Rain']):
        # Ensure there are no NaNs left in 'Rain' before scaling, fill with 0 if any
        df['Rain'].fillna(0, inplace=True)
        scaler = MinMaxScaler()
        df['Rain_Normalized'] = scaler.fit_transform(df[['Rain']])
        print("Numerical 'Rain' column normalized to 'Rain_Normalized'.")
        print('\nDescriptive statistics of the normalized rain:')
        display(df['Rain_Normalized'].describe())
    else:
        print("Warning: 'Rain' column is categorical ('rain', 'no rain') and cannot be directly normalized using MinMaxScaler."
        print("Consider encoding it into numerical format (e.g., binary) if needed for models.")
else:
    print("Warning: 'Rain' column not found. Skipping normalization.")

print('\nFinal preprocessed DataFrame (first 5 rows):')
display(df.head())
```

```
Warning: 'Rain' column is categorical ('rain', 'no rain') and cannot be directly normalized using MinMaxScaler.
Consider encoding it into numerical format (e.g., binary) if needed for models.

Final preprocessed DataFrame (first 5 rows):
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1032.378759	rain
1	27.879734	46.489704	5.952484	4.990053	992.614190	no rain
2	25.069084	83.072843	1.371992	14.855784	1007.231620	no rain
3	23.622080	74.367758	7.050551	67.255282	982.632013	rain
4	20.591370	96.858822	4.643921	47.676444	980.825142	no rain

Task 4 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:

- **Handle missing values in rating, genre.**
- **Encode categorical variables like genre, language.**
- **Remove duplicate movie entries.**
- **Scale rating values between 0 and 1.**

Dataset link:

https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data

Expected Output:

- **A dataset suitable for recommendation systems.**

PROMPT:

Write a Python code using pandas and sklearn to preprocess a the IMDB-Movie-Data.csv. Handle missing values in rating and genre, encode categorical variables like genre and language, remove duplicate entries, and scale the rating column between 0 and 1. The final dataset should be suitable for building a recommendation system.

GIVEN CODE AND OUTPUT:

Preprocessing IMDB Movie Data for Recommendation Systems

This notebook demonstrates the preprocessing steps for the `IMDB-Movie-Data.csv` dataset, preparing it for use in building a movie recommendation system. We will focus on handling missing data, encoding categorical features, removing duplicates, and scaling numerical features.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler, MultiLabelBinarizer

# Load the dataset
try:
    df = pd.read_csv('/content/IMDB-Movie-Data.csv')
    print('Dataset loaded successfully. First 5 rows:')
    display(df.head())
    print('\nDataset Information:')
    df.info()
except FileNotFoundError:
    print("Error: 'IMDB-Movie-Data.csv' not found. Please ensure the file is in the correct directory.")
    df = pd.DataFrame() # Create an empty DataFrame to avoid further errors
```

```
Dataset loaded successfully. First 5 rows:
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	76.0
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	65.0
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	62.0
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	59.0
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	40.0

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Rank                1000 non-null   int64
 1   Title               1000 non-null   object
 2   Genre               1000 non-null   object
 3   Description         1000 non-null   object
 4   Director            1000 non-null   object
 5   Actors              1000 non-null   object
 6   Year                1000 non-null   int64
 7   Runtime (Minutes)   1000 non-null   int64
 8   Rating              1000 non-null   float64
 9   Votes               1000 non-null   int64
 10  Revenue (Millions)  872 non-null    float64
 11  Metascore           936 non-null    float64
dtypes: float64(3), int64(4), object(5)
memory usage: 93.9+ KB
```

1. Handle Missing Values

We will handle missing values for 'Rating' (numerical) by filling them with the mean, and for 'Genre' (categorical) by filling with 'Unknown'.

```python
# Fill missing 'Rating' values with the mean
if 'Rating' in df.columns:
    df['Rating'] = df['Rating'].fillna(df['Rating'].mean())
    print("Missing 'Rating' values filled with mean.")
else:
    print("Warning: 'Rating' column not found. Skipping imputation for Rating.")

# Fill missing 'Genre' values with 'Unknown'
if 'Genre' in df.columns:
    df['Genre'] = df['Genre'].fillna('Unknown')
    print("Missing 'Genre' values filled with 'Unknown'.")
else:
    print("Warning: 'Genre' column not found. Skipping imputation for Genre.")

print('\nDataFrame after handling missing values (first 5 rows):')
display(df.head())
```

```
Missing 'Rating' values filled with mean.
Missing 'Genre' values filled with 'Unknown'.

DataFrame after handling missing values (first 5 rows):
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	76.0
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	65.0
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	62.0
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	59.0
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	40.0

2. Remove Duplicate Entries

Duplicate movie entries can skew analysis and recommendations, so we'll remove them based on all columns.

```python
initial_rows = len(df)
df.drop_duplicates(inplace=True)
duplicate_rows = initial_rows - len(df)
if duplicate_rows > 0:
    print(f"Removed {duplicate_rows} duplicate rows.")
else:
    print("No duplicate rows found.")

print('\nDataFrame after removing duplicates (first 5 rows):')
display(df.head())
```

```
No duplicate rows found.

DataFrame after removing duplicates (first 5 rows):
```

	Rank	Title	Genre	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	Metascore
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	76.0
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	65.0
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	62.0
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	59.0
4	5	Suicide Squad	Action,Adventure,Fantasy	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	40.0

3. Encode Categorical Variables

- **Genre:** Since a movie can have multiple genres, we'll use `MultiLabelBinarizer` for one-hot encoding.
- **Language:** Assuming a 'Language' column exists, we'll use one-hot encoding for it. If not, this step will be skipped.

```python
# One-hot encode 'Genre' column (handling multiple genres per movie)
if 'Genre' in df.columns:
    df['Genre_List'] = df['Genre'].apply(lambda x: x.split(',') if isinstance(x, str) else [])
    mlb = MultiLabelBinarizer()
    genre_encoded = mlb.fit_transform(df['Genre_List'])
    genre_df = pd.DataFrame(genre_encoded, columns=mlb.classes_, index=df.index)
    df = pd.concat([df, genre_df], axis=1)
    df.drop(columns=['Genre', 'Genre_List'], inplace=True)
    print("Genre column one-hot encoded.")
else:
    print("Warning: 'Genre' column not found. Skipping genre encoding.")

# One-hot encode 'Language' column if it exists
# (Assuming a common column name like 'Language', adjust if different)
if 'Language' in df.columns:
    language_encoded = pd.get_dummies(df['Language'], prefix='Language')
    df = pd.concat([df, language_encoded], axis=1)
    df.drop(columns=['Language'], inplace=True)
    print("Language column one-hot encoded.")
else:
    print("Warning: 'Language' column not found. Skipping language encoding.")

print('\nDataFrame after categorical encoding (first 5 rows):')
display(df.head())
```

```
Genre column one-hot encoded.
Warning: 'Language' column not found. Skipping language encoding.

DataFrame after categorical encoding (first 5 rows):
```

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	...	Horror	Music	Musical	Mystery	Romance	Sci-Fi	Sport	Thriller	War	Western
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	...	0	0	0	0	0	1	0	0	0	0
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	...	0	0	0	1	0	1	0	0	0	0
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	...	1	0	0	0	0	0	0	1	0	0
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	...	0	0	0	0	0	0	0	0	0	0
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	...	0	0	0	0	0	0	0	0	0	0

5 rows × 31 columns

4. Scale the 'Rating' Column

We will scale the 'Rating' column to a range between 0 and 1 using `MinMaxScaler`. This is beneficial for many machine learning algorithms.

```python
# Scale 'Rating' column to be between 0 and 1
if 'Rating' in df.columns:
    scaler = MinMaxScaler()
    df['Rating_Scaled'] = scaler.fit_transform(df[['Rating']])
    print(f" 'Rating' column scaled to 'Rating_Scaled' (0-1 range).")
else:
    print("Warning: 'Rating' column not found. Skipping rating scaling.")

print('\nFinal Preprocessed DataFrame (first 5 rows):')
display(df.head())
print('\nDescriptive statistics of the scaled rating:')
display(df['Rating_Scaled'].describe())
```

```
 'Rating' column scaled to 'Rating_Scaled' (0-1 range).

Final Preprocessed DataFrame (first 5 rows):
```

	Rank	Title	Description	Director	Actors	Year	Runtime (Minutes)	Rating	Votes	Revenue (Millions)	...	Music	Musical	Mystery	Romance	Sci-Fi	Sport	Thriller	War	Western	Rating_Scaled
0	1	Guardians of the Galaxy	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	2014	121	8.1	757074	333.13	...	0	0	0	0	1	0	0	0	0	0.873239
1	2	Prometheus	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	2012	124	7.0	485820	126.46	...	0	0	1	0	1	0	0	0	0	0.718310
2	3	Split	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	2016	117	7.3	157606	138.12	...	0	0	0	0	0	0	1	0	0	0.760563
3	4	Sing	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	2016	108	7.2	60545	270.32	...	0	0	0	0	0	0	0	0	0	0.746479
4	5	Suicide Squad	A secret government agency recruits some of th...	David Ayer	Will Smith, Jared Leto, Margot Robbie, Viola D...	2016	123	6.2	393727	325.02	...	0	0	0	0	0	0	0	0	0	0.605634

5 rows × 32 columns

```
Descriptive statistics of the scaled rating:
```

	Rating_Scaled
count	1000.000000
mean	0.679324
std	0.133159
min	0.000000
25%	0.605634
50%	0.690141
75%	0.774648
max	1.000000

```
dtype: float64
```

This dataset is now preprocessed and ready for building a movie recommendation system!